

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Arquitetura de Software
Parte 01

Arquitetura de Software

- Arquitetura preocupa-se com **“projeto em mais alto nível”**.
- O foco deixa de ser a organização e interfaces de classes individuais e
 - Passa a ser em **unidades de maior tamanho**, sejam elas **pacotes, componentes, módulos, subsistemas, camadas ou serviços** — o nome não importa tanto neste primeiro momento.

Arquitetura de Software

- De forma genérica, os tais termos devem ser entendidos como **conjuntos de classes relacionadas.**
- **Padrões (estilos) arquiteturais** propõem uma **organização de mais alto nível** para sistemas de software, incluindo seus principais **módulos** e as relações entre eles.
- Essas **relações** definem, por exemplo, **que um módulo A** pode (ou não pode) **usar os serviços de um módulo B.**

Arquitetura de Software

- Então.
 - O projeto de arquitetura está preocupado com a **compreensão de como um sistema deve ser organizado** e com a estrutura geral desse sistema.
 - No modelo do processo de **desenvolvimento de software**, o *projeto de arquitetura é o primeiro estágio no processo de projeto de software.*

Tipos de Arquitetura de Software



Arquitetura de Software

- **Arquitetura em camadas**

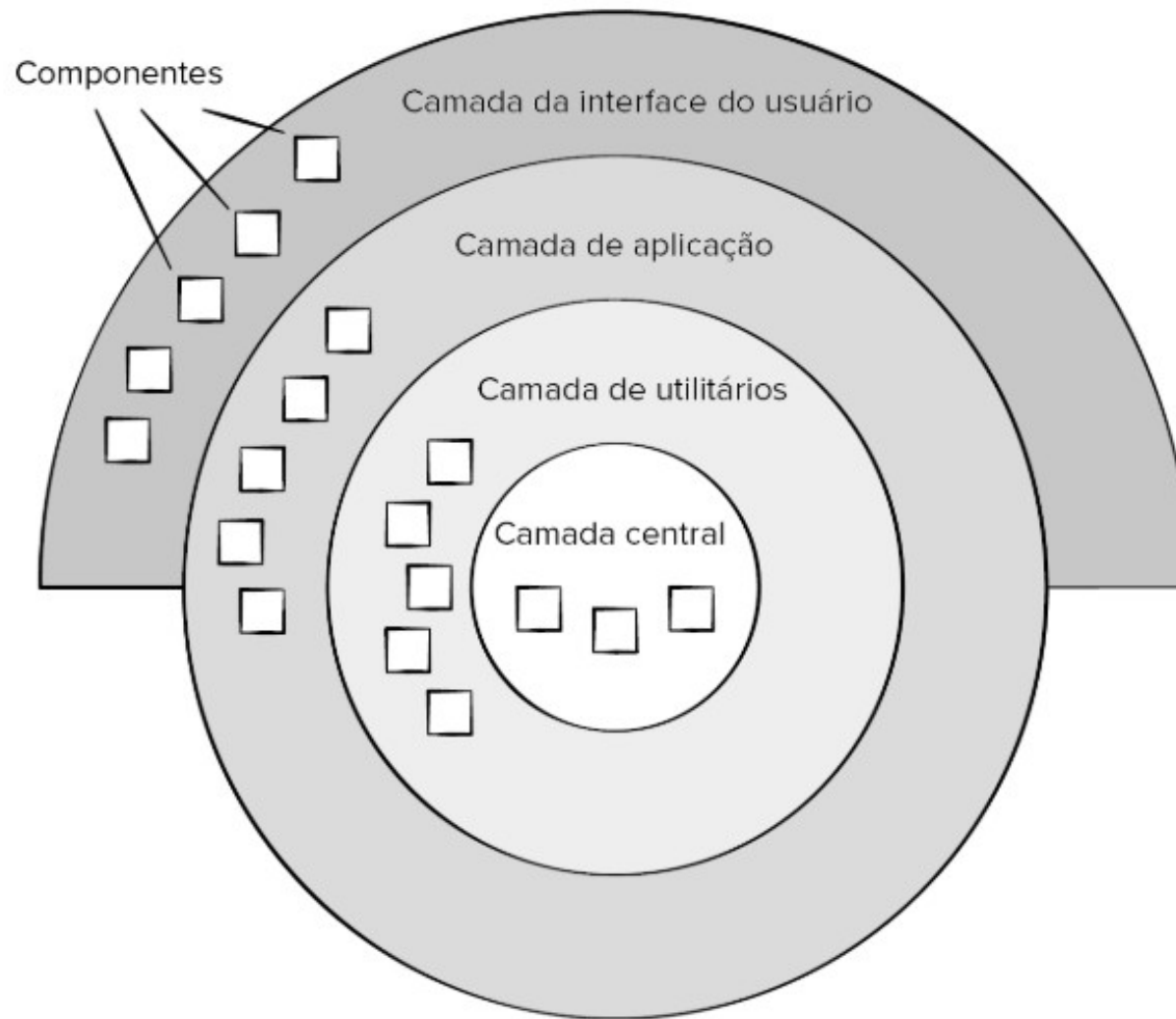
- É um dos padrões arquiteturais mais usados **(camadas são componentes ou módulos que implementam diferentes serviços),**
- desde que os primeiros sistemas de software de maior porte foram construídos nas décadas de 60 e 70.
- Em sistemas que seguem esse padrão, **as classes são organizadas em módulos** de maior tamanho, chamados de **camadas**.

Arquitetura de Software

- As camadas são dispostas de forma hierárquica, como em um bolo.
- Assim, uma camada somente pode usar serviços — isto é, chamar métodos, instanciar objetos, estender classes, lançar exceções — da camada imediatamente inferior.
 - Divide o sistema em camadas horizontais, cada uma com uma função específica.
 - As camadas superiores dependem das inferiores, promovendo a organização e a reutilização.

Arquitetura de Software

- Arquitetura em camadas.



Arquitetura de Software

- Arquiteturas em camadas são **muito usadas na implementação de protocolos de rede.**
- Por exemplo, **HTTP** é um protocolo de aplicação, que usa serviços de um protocolo de transporte; por exemplo, TCP).
- Por sua vez, TCP usa serviços de um protocolo de rede; por exemplo, IP.
- Finalmente, a camada IP usa serviços de um protocolo de comunicação; por exemplo, Ethernet.
 - **Particiona a complexidade** envolvida no desenvolvimento de um sistema em **componentes menores (as camadas).**

Arquitetura de Software

- **Disciplina as dependências** entre essas camadas:
 - A **camada n** somente pode usar serviços da **camada n-1**.
 - Isso ajuda no **entendimento**, manutenção e evolução de um sistema.
 - Torna-se mais fácil **trocar uma camada por outra** (por exemplo, mudar de TCP para UDP).

Arquitetura de Software

- Fica **mais fácil também o reúso** de uma camada por mais de uma camada superior.
 - Por exemplo, a **camada de transporte** pode ser **usada por vários protocolos** de aplicação, como HTTP, SMTP, DHCP, etc.
 - Benefícios de uma estrutura hierárquica são mais importantes ainda em projetos de maior porte.

Arquitetura de Software

- Arquitetura em **Três Camadas**
 - Esse tipo de arquitetura é **comum na construção de sistemas de informação corporativos.**
 - As aplicações eram **monolíticas.**
 - Com o avanço nas **tecnologias de rede e de hardware,** foi possível migrar esses sistemas de **mainframes** para outras plataformas.

Arquitetura de Software

- Modelo em três camadas (3-Tier),
 - derivado do modelo 'n' camadas, recebe esta denominação quando um sistema cliente-servidor é desenvolvido retirando-se a camada de negócio do lado do cliente.

Arquitetura de Software

- Com o advento da Internet, houve um movimento para separar a lógica de negócio da interface com o usuário.
- A ideia é que os usuários da web possam acessar as mesmas aplicações sem ter que instalar estas aplicações em suas máquinas locais.
 - Como a lógica do aplicativo, inicialmente contida no cliente rico, não mais reside na máquina do usuário, este tipo de cliente passou a ser chamado de **cliente pobre ou magro (Thin Client)**.

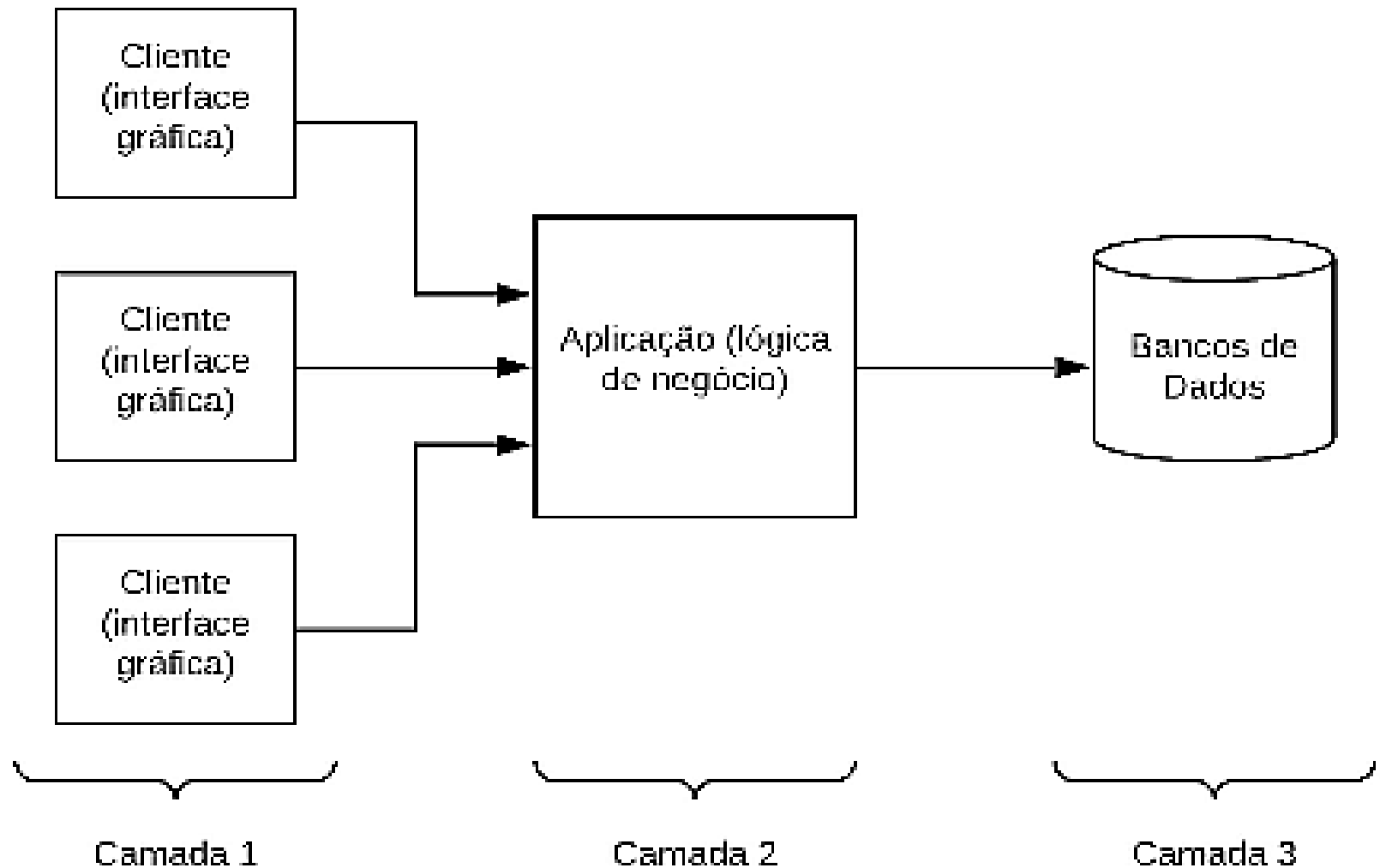
Arquitetura de Software

- Neste modelo o aplicativo é movido para o servidor e um navegador web é usado como um cliente magro.
- O aplicativo é executado em servidores web com os quais o navegador web se comunica e gera o código HTML para ser exibido no cliente.

Arquitetura de Software

- As **três camadas** dessa arquitetura são as seguintes:
 - **Interface com o Usuário**, também chamada de camada de **apresentação**, é responsável por toda interação com o usuário.
 - Ela trata **tanto da exibição de informação**, como da coleta e processamento de entradas e eventos de interfaces, tais como cliques em botões, marcação de texto, etc.
 - **Lógica de Negócio**, também conhecida como camada de aplicação, implementa as regras de negócio do sistema.
 - **Banco de Dados**, que armazena os dados manipulados pelo sistema.

Arquitetura de Software



Arquitetura de Software

- Desta forma:
 - No modelo 3 camadas, a lógica de apresentação esta separada em sua **própria camada lógica e física**.
 - A **separação em camadas lógicas** torna os sistemas mais flexíveis, permitindo que as partes possam ser alteradas de forma independente.
 - O modelo de 3 camadas tornou-se a arquitetura padrão para sistemas corporativos com base na Web.

Arquitetura de Software

- Normalmente, uma arquitetura em três camadas é uma arquitetura **distribuída**.
- Isto é, a camada de interface executa na máquina dos clientes.
- A camada de negócio executa em um servidor, muitas vezes chamado de servidor de aplicação.
 - A camada de aplicação pode ter diversos módulos, incluindo uma fachada, para facilitar o acesso ao sistema pelos clientes, e um módulo de persistência, com a função de isolar o banco de dados dos demais módulos.
- E, por fim, temos o servidor de banco de dados.

Arquitetura de Software

- Sistemas em **duas camadas**.
 - Nesses casos, as **camadas de interface e de aplicação são unidas em uma única camada**, que executa no cliente.
 - A camada restante continua sendo o banco de dados.
 - A desvantagem de arquiteturas em duas camadas é que **todo o processamento ocorre nos clientes**, que, portanto, devem ter um maior poder de computação.

Arquitetura de Software

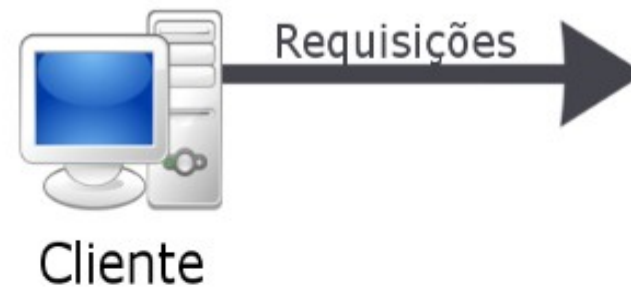
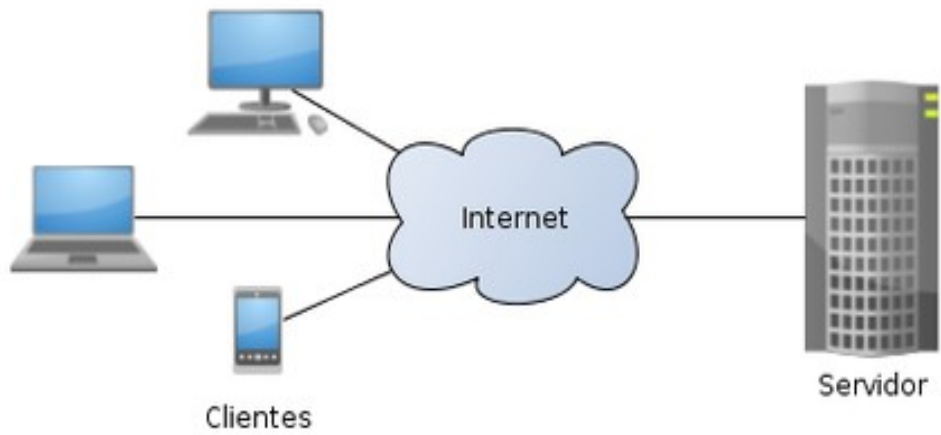
- Nesta estrutura, a base de dados foi colocada em uma máquina específica, separada das máquinas que executavam as aplicações.
 - Nessa abordagem, temos aplicativos instalados em estações clientes contendo toda a lógica da aplicação **(clientes ricos ou gordos)**.
 - Um grande problema neste modelo é o gerenciamento de versões pois, para cada alteração, os aplicativos precisam ser atualizados em todas as máquinas clientes.

Arquitetura de Software

- A arquitetura de duas camadas é a arquitetura de cliente-servidor original, composta por
 - uma camada de apresentação e
 - uma camada de dados.
- A lógica de negócios reside na camada de apresentação, na camada de dados ou em ambas.
- Camadas são componentes de software considerando separação lógica, que possibilita separação física.

Arquitetura de Software

- A arquitetura cliente servidor
 - é uma arquitetura de aplicação distribuída,
 - ou seja, na rede existem os fornecedores de recursos ou serviços a rede, que são chamados de servidores, e existem os requerentes dos recursos ou serviços, denominados clientes.



Arquitetura de Software

- Padrão arquitetural **MVC (Model-View-Controller)**
- As classes de um sistema devem ser organizadas em três grupos:
 - **Visão**: classes responsáveis pela apresentação da interface gráfica do sistema, incluindo janelas, botões, menus, barras de rolagem, etc.

Arquitetura de Software

- **Controladoras**: classes que tratam e interpretam eventos gerados por dispositivos de entrada, como mouse e teclado.
 - Controladoras **podem solicitar** uma alteração no estado do **Modelo ou da Visão**.
 - Quando o **usuário clica em um botão**, uma classe Controladora deve **capturar esse evento e chamar um método do Modelo**.

Arquitetura de Software

- **Modelo**: classes que tratam **os dados manipulados pela aplicação** e que têm a ver com o **domínio** do sistema em construção, além das regras de negócio (**requisitos funcionais**).
 - Assim, classes de modelo **não têm qualquer conhecimento ou dependência** para classes de Visão e Controladoras.
 - Além de dados, **classes de modelo podem conter métodos** que alteram o estado dos objetos de domínio.

Arquitetura de Software

- Portanto, em uma arquitetura MVC, a **interface gráfica é formada por objetos de visão e por controladores.**
- Porém, em muitos sistemas **não existe uma distinção clara entre Visão e Controladores.**

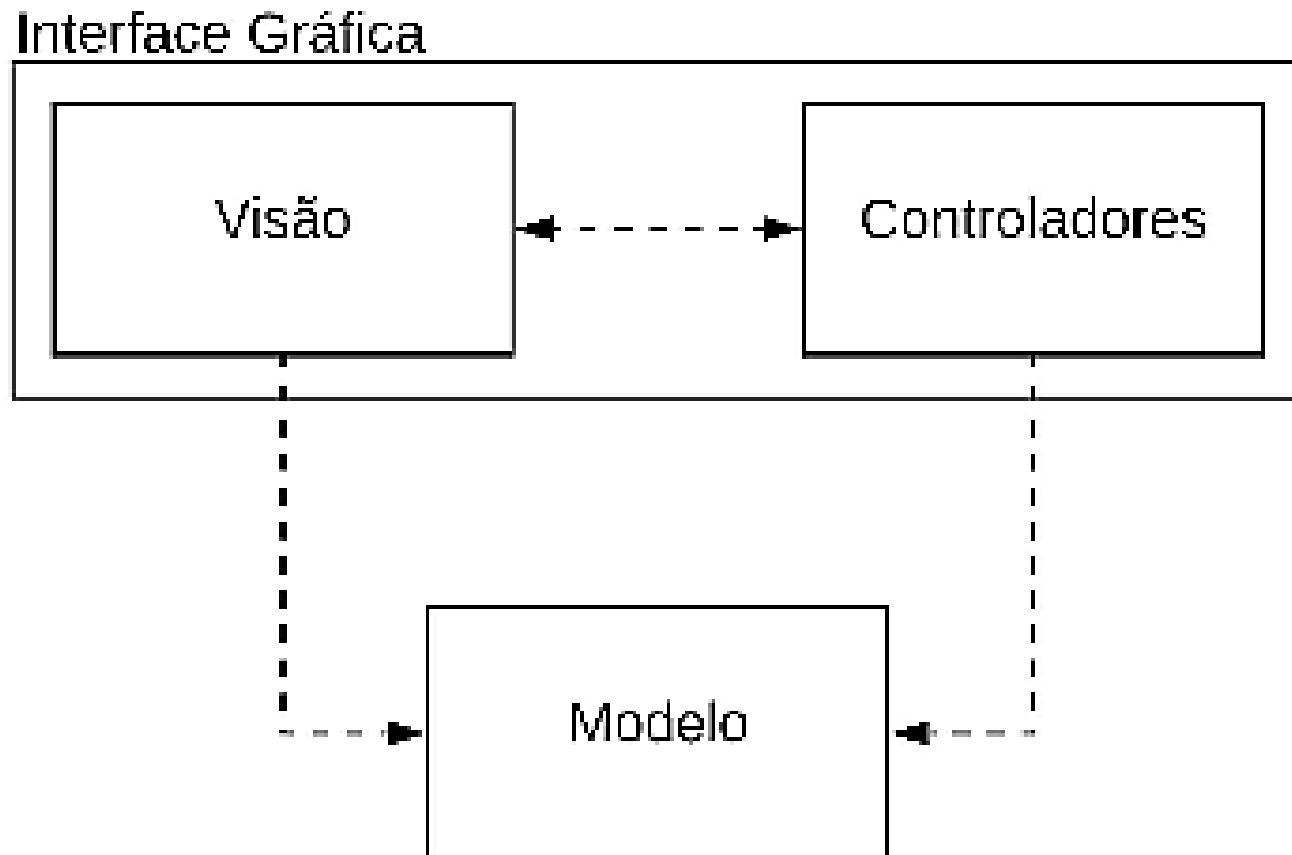
MVC = (Visão + Controladores) + Modelo = Interface Gráfica + Modelo

Arquitetura de Software

- A interface depende do modelo.
- As **classes do modelo** não tem dependências para classes da interface.
- A **interface é uma observadora** do modelo (observer),
 - Quando os estados dos objetos do modelo são alterados, deve-se atualizar automaticamente a interface.

Arquitetura de Software

- **Pode-se adicionar uma nova visão ou alterar uma exibição existente** sem quaisquer alterações nos dados subjacentes do modelo



Arquitetura de Software

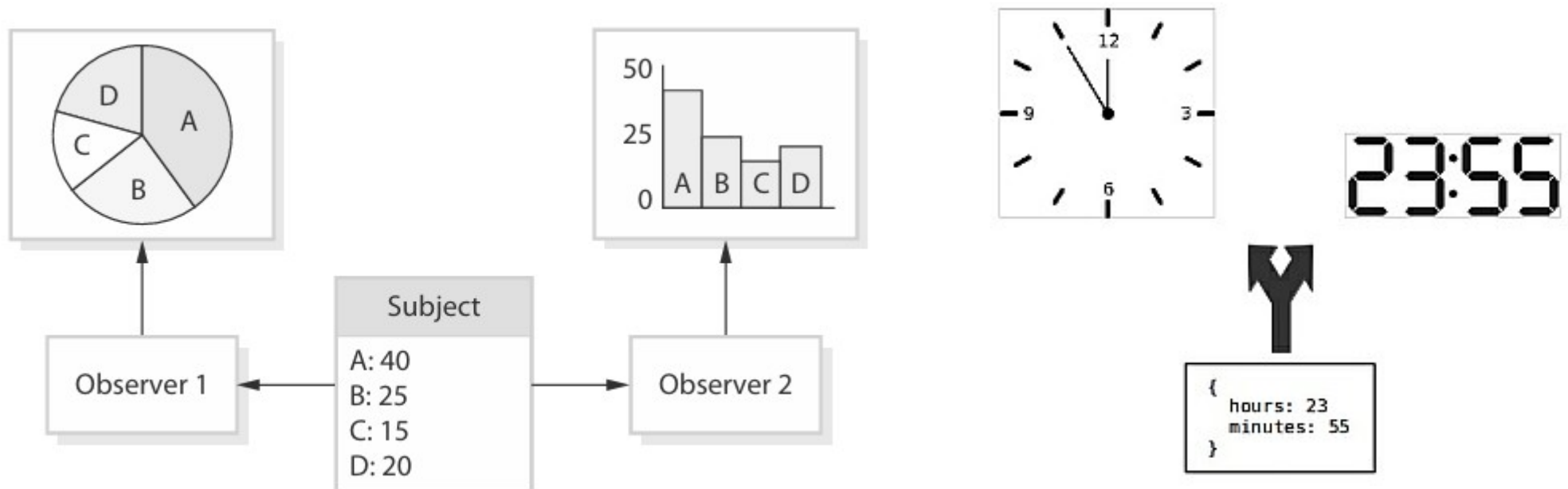
- Vantagens:
 - Permite **especializar o trabalho de desenvolvimento**, com especialistas em interfaces (desenvolvedores front-end).
 - Desenvolvedore de modelo (back-end) não precisam conhecer e implementar código de interface.
 - Favorece **testabilidade**: é mais fácil **testar objetos de modelo isoladamente** da implementação de interfaces.
 - Permite que **classes de modelo** sejam usadas por **diferentes visões**.

Arquitetura de Software

- Desta forma, a parte mais importante de MVC **está na separação entre código de interface com usuário e a lógica do domínio**, o modelo.
- **Classes de domínio não incluem código visual**, apenas a lógica de negócios.
- Isso **separa duas partes complexas** de sistemas em partes que são mais fáceis de modificar e reutilizar.
- **Também permite várias apresentações** da mesma lógica de negócios.

Arquitetura de Software

- Sistema MVC com mais de uma visão
 - solução que **possibilita a existência de várias interfaces com o usuário que podem ser modificadas sem que haja a necessidade da alteração das regras de negócio**, proporcionando assim muito mais flexibilidade e oportunidades de reuso das classes.



- Resumindo...

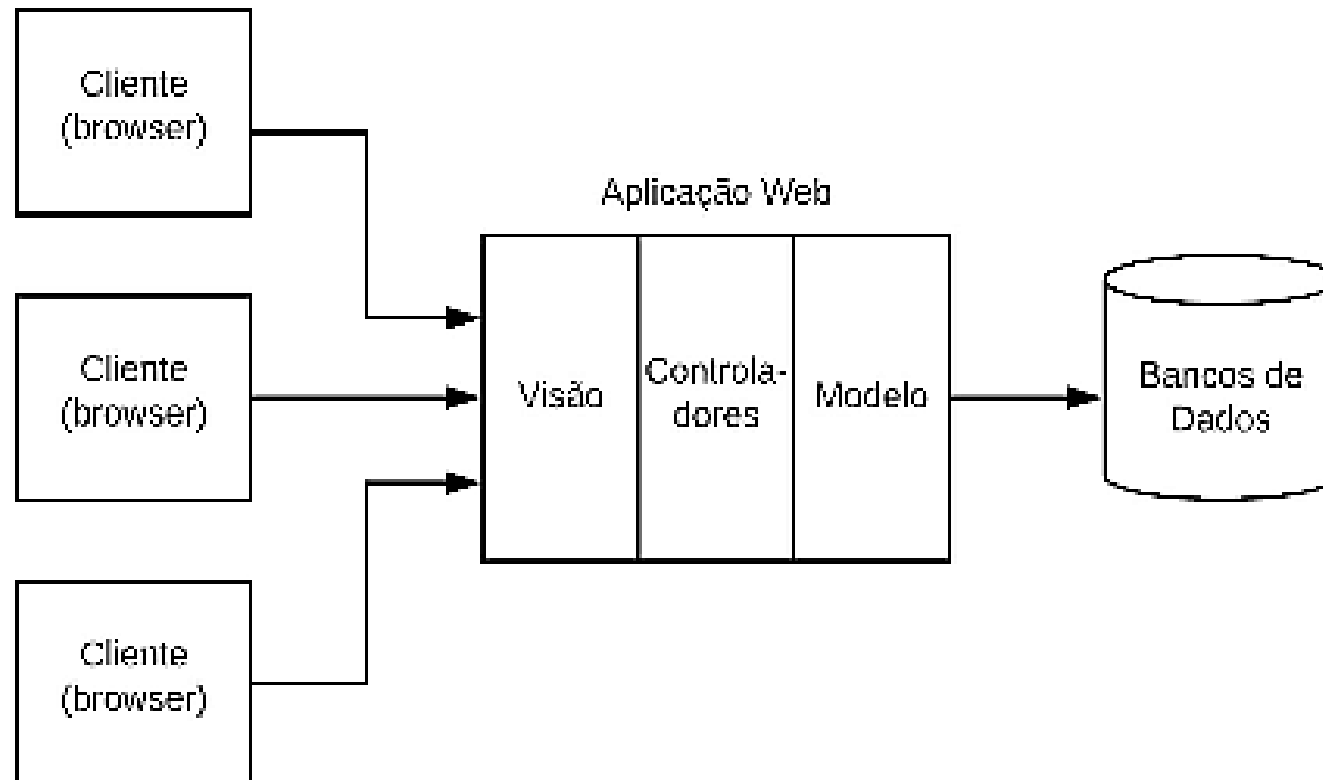
Nome	MVC (Modelo-Visão-Controlador)
Descrição	Separa a apresentação e a interação dos dados do sistema. O sistema é estruturado em três componentes lógicos que interagem entre si. O componente Modelo gerencia o sistema de dados e as operações associadas a esses dados. O componente Visão define e gerencia como os dados são apresentados ao usuário. O componente Controlador gerencia a interação do usuário (por exemplo, teclas, cliques do mouse etc.) e passa essas interações para a Visão e o Modelo.
Quando é usado	É usado quando existem várias maneiras de se visualizar e interagir com dados. Também quando são desconhecidos os futuros requisitos de interação e apresentação de dados.
Vantagens	Permite que os dados sejam alterados de forma independente de sua representação, e vice-versa. Apoia a apresentação dos mesmos dados de maneiras diferentes, com as alterações feitas em uma representação aparecendo em todas elas.
Desvantagens	Quando o modelo de dados e as interações são simples, pode envolver código adicional e complexidade de código.

Arquitetura de Software

- Frameworks MVC
 - Spring (Java)
 - Rails (Ruby) – Ruby on Rails
 - Django (Python)
 - CakePHP (PHP)
- Adaptaram o conceito de MVC para web.
- Forçam o desenvolvimento do sistema web em uma arquitetura MVC.

Arquitetura de Software

- Arquitetura MVC para web.



Arquitetura de Software

- Exemplo – Spring MVC

- Modelo

```
public class Cliente {  
    private String nome;  
    private String cpf;  
    private String rg;  
    //getters e setter  
    //operações, regras de negócio  
    ...  
}
```

Controle

@Controller

public class ClienteController {

@RequestMapping(value = "/cliente", method = RequestMethod.GET)

public ModelAndView cliente() {

return new ModelAndView("cliente", "command", new Cliente());

}

@RequestMapping(value = "/addCliente", method = RequestMethod.POST)

public String adicionarCliente(

@ModelAttribute("SpringWeb") Cliente cliente, ModelMap model,

HttpServletRequest request) {

...

@RequestMapping(value = "/listaClientes", method = RequestMethod.GET)

public String listarClientes(...

}

Visão

cliente.jsp

```
<html>
<head>
  <title>Cadastro de Cliente</title>
</head>
<body>
<h2>Formulário de Cadastro de Cliente</h2>
<form:form method="POST" action="/cliente/addCliente">
  <table>
    <tr>
      <td><form:label path="nome">Nome</form:label></td>
      <td><form:input path="nome" /></td>
    </tr>
    <tr>
      <td><form:label path="cpf">CPF:<
        /form:label></td>
      <td><form:input path="cpf" /></td>
    </tr>
    <tr>
      <td><form:label path="rg">RG:<
        /form:label></td>
      <td><form:input path="rg" /></td>
    </tr>
    ...
  </form:form>
  <a href="/cliente/listaClientes">Lista</a>
```

Arquitetura de Software

- Exemplo completo disponível em

<https://www.devmedia.com.br/java-spring-mvc-criando-aplicacoes-web-em-java/31521>

Arquitetura de Software

- As arquiteturas descritas aqui são **monolíticas**.
 - A **arquitetura monolítica** é um modelo tradicional de desenvolvimento de software onde **todas as funcionalidades de um aplicativo são agrupadas em um único sistema**, executando todas as tarefas em uma **única base de código**.
 - É como um grande bloco (monolito) onde tudo está interligado e depende um do outro.
 - **Existe uma separação lógica do código, mas não existe uma separação física do código**, como em arquiteturas distribuídas (ex. arquitetura de microserviços).

Arquitetura de Software

- **Base de código única**: Toda a lógica e funcionalidades do aplicativo estão contidas em uma única base de código.
- **Implantação única**: A aplicação monolítica é geralmente implantada como um único arquivo ou conjunto de arquivos.
- **Simplicidade**: É mais simples de desenvolver e entender, especialmente para aplicações menores e projetos de menor complexidade.
- **Facilidade de depuração**: A depuração pode ser mais fácil, pois tudo está contido em um único lugar.

Arquitetura de Software

- **Manutenção e atualização**: Pequenas alterações em uma parte do sistema podem exigir a recompilação e reimplementação de todo o sistema, o que pode ser demorado e arriscado.
- **Risco de falhas**: Se um componente falhar, pode afetar todo o sistema, comprometendo a disponibilidade do aplicativo.
- **Flexibilidade limitada**: É mais difícil adicionar novas tecnologias ou frameworks sem afetar o sistema como um todo.
- **Desafios de escalabilidade**: Escalar um sistema monolítico pode ser difícil, especialmente se algumas partes precisarem de mais recursos do que outras.

Arquitetura de Software

- **Escalabilidade**

- Com o modelo **Multicamadas (três camadas ou n-tier)**,
- é possível ter a mesma regra de negócio dividida entre vários servidores **através do balanceamento de carga (load balance)**, ou seja, quando algum deles ficar sobrecarregado o outro entrará em ação para ajudá-lo.
 - Se ocorrer algum problema com algum **servidor e este não puder mais responder às requisições (ficar off-line, por exemplo)**, outro servidor poderá entrar em seu lugar.

Arquitetura de Software

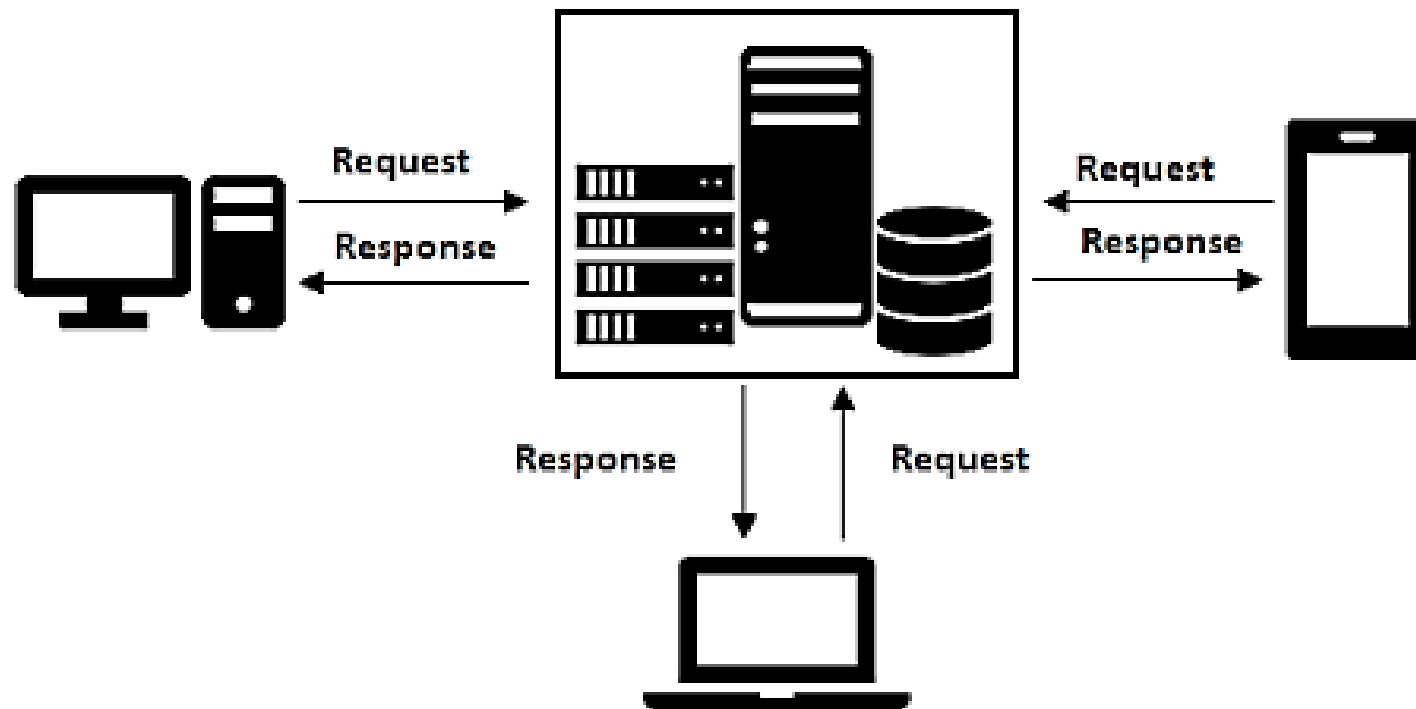
- **Escalabilidade** em TI é a capacidade de um sistema, aplicação ou infraestrutura de se adaptar a um aumento ou diminuição na demanda,
 - com o mesmo padrão de qualidade do serviço oferecido
 - Quando falamos de um servidor ou storage, a **escalabilidade** refere-se à capacidade desse sistema em aumentar o número de transações processadas por segundo
 - conforme são adicionados mais recursos, como CPU, RAM ou novas unidades de armazenamento.

Arquitetura de Software

- Por que a arquitetura é importante?
 - A arquitetura de software fornece uma representação que facilita a comunicação entre todos os envolvidos.
 - A arquitetura destaca desde o início as decisões de projeto que terão um profundo impacto no trabalho de engenharia de software que se segue.
 - A arquitetura constitui um modelo relativamente pequeno de como os componentes do sistema estão estruturados e trabalham em conjunto.

Arquitetura de Software

- Exercícios
- Os padrões arquiteturais de software estão ligados a aspectos amplos sobre o funcionamento e a operação da aplicação. Dessa forma, envolve fatores como: hardware, disponibilidade, tecnologias de comunicação, portabilidade e riscos. Analise a imagem a seguir:



- Trata-se da seguinte arquitetura de software:
- a) Pipeline.
- b) Peer to Peer.
- c) Cliente-Servidor.
- d) MVC (Model-View-Controller).

- RESPOSTA

- LETRA C

- No que se refere à arquitetura cliente-servidor, julgue o item
- Uma das desvantagens desse tipo de arquitetura é que a comunicação cliente-servidor não é transparente para o usuário final
- Errado
- Certo

- RESPOSTA

- ERRADO, a comunicação é transparente para o usuário final, e isto é uma grande vantagem.
 - Ou seja, o usuário não conhece os protocolos ou como a comunicação acontece.

- No que se refere à arquitetura cliente-servidor, julgue o item
- Além das diversas outras funções do servidor, cabe a ele iniciar a comunicação de uma transação com o cliente.
- Errado
- Certo

- RESPOSTA

- ERRADO, a comunicação é iniciada através de uma requisição do cliente.

- Arquiteturas em duas camadas separam as entidades em camadas de apresentação e de armazenamento, as quais podem estar, ou não, no mesmo computador.
- Errado
- Certo

- RESPOSTA

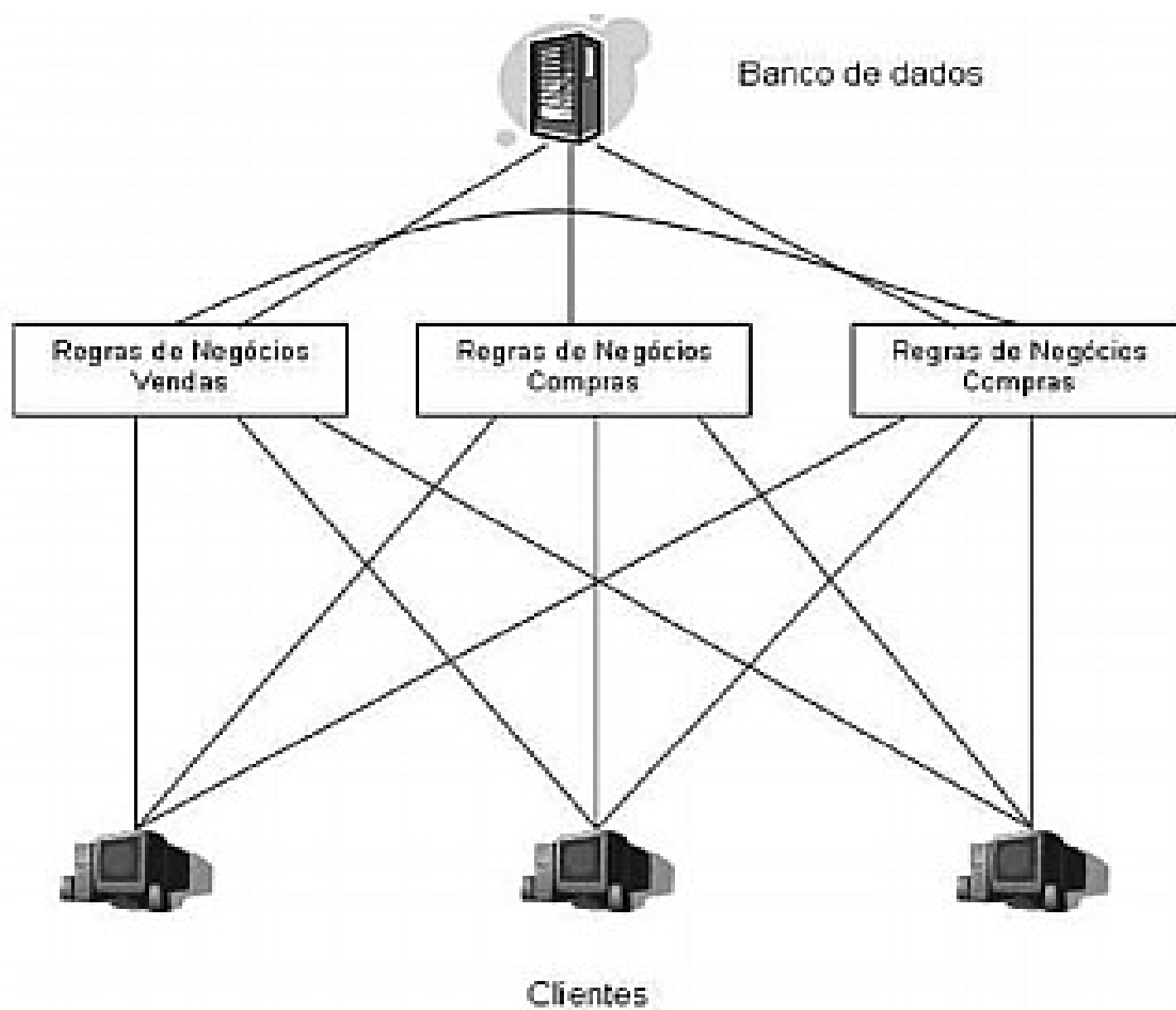
- ERRADO, A arquitetura de duas camadas é a arquitetura de cliente-servidor original, composta por uma camada de apresentação e uma camada de dados.
 - O cliente, que interage diretamente com o usuário, possui uma interface que envia solicitações ao servidor, responsável pelo armazenamento e processamento dos dados.
 - A lógica de negócios pode estar presente tanto no cliente quanto no servidor, ou em ambos.

- Na arquitetura de 3 camadas, entrada do usuário, verificação, lógica de negócio e acesso a banco de dados estão todos presentes em um mesmo lugar, onde essas camadas são organizadas.
- Errado
- Certo

- RESPOSTA

- ERRADO, A arquitetura em 3 camadas, também conhecida como arquitetura multicamadas, divide uma aplicação em três camadas lógicas distintas:
 - apresentação, lógica de negócios e dados.
 - Essa separação permite maior modularidade, escalabilidade e flexibilidade no desenvolvimento e manutenção da aplicação.

- Na figura a seguir, na porção superior está localizado o servidor de banco de dados. A comunicação com os servidores de aplicação é feita através de algum protocolo de rede (TCP/IP, por exemplo) e o acesso aos dados é realizado por meio da linguagem SQL (Structured Query Language). Na parte inferior estão as estações clientes. A comunicação destas com a camada intermediária é feita através da utilização de interfaces. Este é basicamente o esquema de comunicação desta arquitetura e não pode ser alterado. Nesse contexto, assinale a alternativa que apresenta as vantagens do desenvolvimento em multicamadas.



- A respeito dos conceitos de camadas (tiers), de thin client e fat clients na arquitetura web, assinale a opção correta.
- a) A grande vantagem da thin client é a distribuição uniforme de carga de processamento de domínio e de regra de negócio entre as camadas (tiers).
- b) Tanto na thin client quanto na fat client as regras de negócios devem ficar armazenadas em stored procedures na camada de persistência, a fim de permitir maior independência em relação ao SGBD.
- c) Os clientes (desktops) são classificados como fat clients por permitirem compatibilidade com inúmeros tipos de equipamentos, incluindo-se dispositivos móveis.
- d) Na arquitetura tradicional, há três tiers: o cliente remoto, o servidor web e o servidor de banco de dados.

- RESPOSTA
 - LETRA D

- Na arquitetura em 3 camadas, a aplicação cliente apresenta grande diminuição de código e todo o trabalho de instalação é bastante reduzido, possuindo somente uma configuração para o cliente ter acesso à camada intermediária. Por esta razão, há diminuição de custos, uma vez que não existe necessidade de upgrade nas estações clientes que apresentam poucos recursos de hardware ou que são computadores antigos. Assinale a alternativa que identifica corretamente o tipo de vantagem do desenvolvimento em multicamadas, proposto no texto.
- a) Escalabilidade.
- b) Modularização.
- c) Criptografia.
- d) Facilidade de redistribuição.
- e) Clientes leves (thin-clients).

- RESPOSTA
 - LETRA E

- Com o modelo Multicamadas, é possível ter a mesma regra de negócio dividida entre vários servidores através do balanceamento de carga, ou seja, quando algum deles ficar sobrecarregado o outro entrará em ação para ajudá-lo. Se ocorrer algum problema com algum servidor e este não puder mais responder às requisições (ficar off-line, por exemplo), outro servidor poderá entrar em seu lugar. Assinale a alternativa que nomeia corretamente tal propriedade e vantagem da arquitetura Cliente Servidor Multicamadas.

a) Escalabilidade.

b) Eficiência.

c) Confiabilidade.

d) Reusabilidade.

e) Flexibilidade.

- RESPOSTA
 - LETRA A

- **Exercícios**

- O Modelo MVC é uma técnica de modelagem adotada em sistemas interativos, que agrupa os seus componentes sob as perspectivas Model, View e Controller (Buschmann et al., 1996). Nesse sentido, analise:
- I. podem ser criados vários componentes definindo várias Views para apresentação diferenciada de um mesmo dado. Estas Views podem ser usadas simultaneamente, permitindo que um mesmo dado seja apresentado de várias maneiras ao mesmo tempo. As Views que serão utilizadas podem ser selecionadas em tempo de execução;
- II. a separação em camadas do MVC permite a troca dos componentes nas camadas View e Controller sem afetar a camada Model. As camadas View e Controller definem a interface do sistema, que poderá ser então substituída completamente em tempo de execução sem afetar a camada de processamento de dados principal;
- III. o alto grau de acoplamento entre as camadas View e Controller repercute na camada de processamento principal (Model), mas não diminui a portabilidade do sistema, uma vez que as interfaces entre as camadas podem ser alteradas para se adequarem aos elementos de uma nova plataforma.

- É correto o que consta APENAS em:
- (A) I e II.
- (B) I e III.
- (C) II.
- (D) II e III.
- (E) III.

- RESPOSTA

- LETRA A

- Analise a Figura 1 abaixo e assinale a alternativa correta

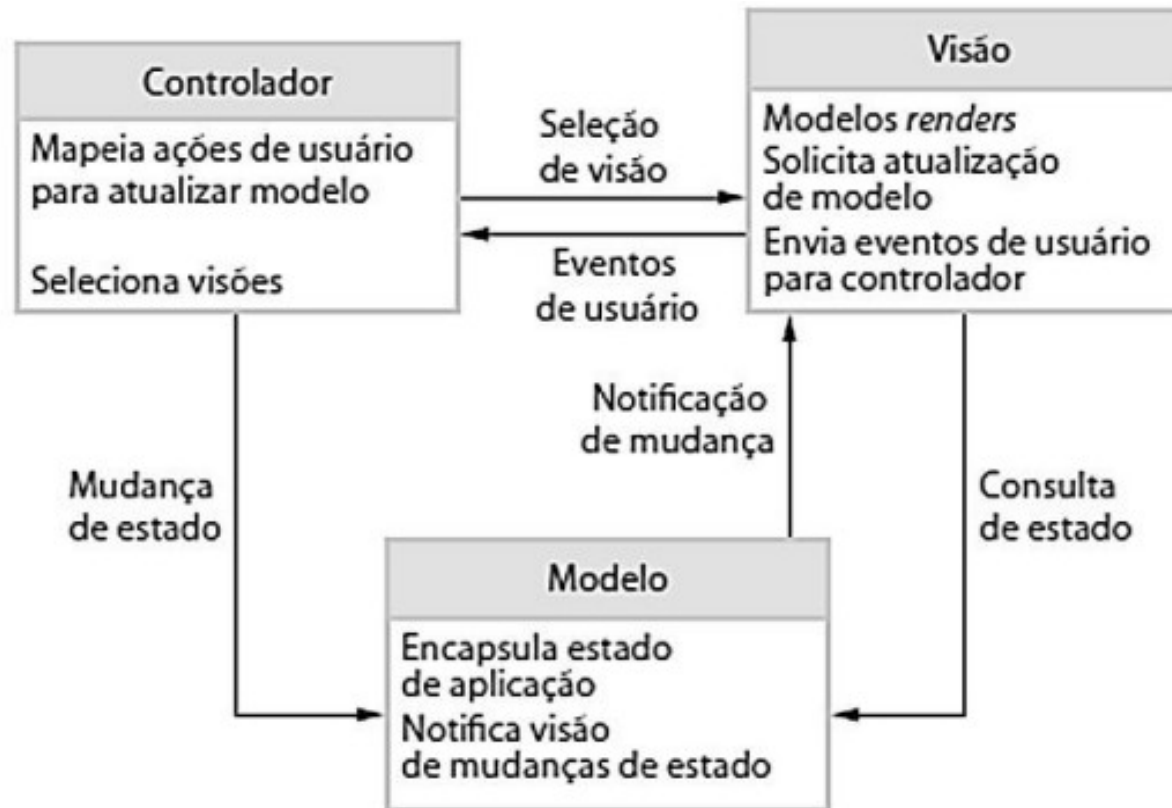


Figura 1 (SOMMERVILLE, 2007).

- (A) Especificação de um diagrama de classe.
- (B) Trata-se de uma arquitetura MVC (Model-View-Controller).
- (C) São os tipos de padrão de Design Pattern: modelo, controlador e visão.
- (D) Refere-se a uma arquitetura de projetos SCRUM.
- (E) Faz referência a um metamodelo UML de classificação.

- RESPOSTA

- LETRA B

- A divisão em camadas da arquitetura MVC permitiu o uso de diversos padrões de projeto de forma simples, e, com toda a padronização obtida, aliada a ferramentas como injeção de dependências e inversão de controle, ocorreu o surgimento de diversos frameworks, voltados para as diferentes camadas da arquitetura. Para a camada Model, um padrão essencial é o:
 - (A) Front Controller;
 - (B) Observer;
 - (C) DAO;
 - (D) Facade;
 - (E) Composite.

- RESPOSTA
- LETRA B
 - Neste caso, o model é o sujeito, e a view é a observadora. Modificações nos dados do modelo produzem notificações nas diferentes views.
 - Como o padrão observer usa polimorfismo através de um supertipo, não existe acoplamento entre o modelo e uma view específica.

- A arquitetura MVC (Model-View-Controller), um modelo de arquitetura de software, é aplicada à divisão ou abstração de tarefas em camadas independentes que, através de trocas de requisições e mensagens, se interagem de forma a organizar o código para que sua manutenção, encapsulamento e reúso fiquem mais dinâmicos e simples, pois permite uma melhor separação de responsabilidades e de distribuição de tarefas. Diante do exposto, analise as afirmativas a seguir.
- I. Model: camada que contém a aplicação física, e de sua responsabilidade apresentar a interface gráfica do sistema aos usuários.
- II. Controller: define o comportamento da aplicação; é como um gerenciador da interface do usuário, interpretando suas ações e que se comunica por meio de eventos disparados pela interação do usuário com o Modelo no padrão MVC para atualizar dados à View. A função desse padrão de arquitetura é tornar possível a separação entre as camadas.
- III. View: camada de apresentação com usuário, ou seja, são classes responsáveis pela apresentação da interface gráfica do sistema, onde são realizadas a entrada de dados e a visualização das respostas processadas.

- Está correto o que se afirma em:
- (A) I, II e III.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.

- RESPOSTA
- LETRA D

- Julgue o item a seguir, a respeito de desenvolvimento web em Java e de interoperabilidade de sistemas.
- O desenvolvimento de aplicações disponibilizadas em múltiplas interfaces de usuário é facilitado pela utilização do modelo MVC, pois a principal característica deste é a existência de uma camada de regra de negócios totalmente isolada da interface de usuário.
- Alternativas
- Certo
- Errado

- RESPOSTA
- CERTO

- Um Técnico observou que o padrão MVC define regras de como seus componentes devem se comunicar. Assim, está correto que:
- (A) Controllers não podem interagir com Views e Views podem interagir diretamente com o Model.
- (B) Controllers não podem se comunicar com Models e Views podem interagir diretamente com outras Views.
- (C) Os usuários não podem interagir diretamente com o Model e Controllers podem se comunicar com o Model.
- (D) Os usuários podem interagir diretamente com os Controllers e Views podem interagir diretamente com o Model.
- (E) Controllers não podem interagir com Views e Controllers podem se comunicar com o Model.

- RESPOSTA
- LETRA C

- Quanto ao Modelo Visão-Controlador (MVC), conforme **Sommerville (2011)**, analise as afirmativas a seguir e assinale a alternativa correta:
- (1) O componente Controlador define e gerencia como os dados são apresentados ao usuário.
- (2) O componente Visão gerencia o sistema de dados e as operações associadas a esses dados.
- (A) As duas afirmativas são verdadeiras
- (B) A afirmativa (1) é verdadeira, e a (2) é falsa
- (C) A afirmativa (2) é verdadeira, e a (1) é falsa
- (D) As duas afirmativas são falsas

- RESPOSTA
- LETRA D

- Sobre o padrão MVC (Model-View-Controller), assinale a alternativa que apresenta uma das funções da camada controladora (Controller).
- (A) Preparar dados do modelo.
- (B) Apresentar conteúdos ao usuário.
- (C) Acessar fontes de dados externas.
- (D) Gerenciar solicitações do usuário.
- (E) Encapsular objetos de conteúdo.

- RESPOSTA
- LETRA D

- Um Analista em TI necessita desenvolver um projeto otimizando o reuso das classes. Para tanto, ele está analisando uma solução que possibilita a existência de várias interfaces com o usuário que podem ser modificadas sem que haja a necessidade da alteração das regras de negócio, proporcionando assim muito mais flexibilidade e oportunidades de reuso das classes. O modelo de arquitetura padrão que atende essa necessidade porque contempla a divisão do projeto em camadas muito bem definidas é o:

- a) MVC.
- b) SOAP.
- c) SPRINT.
- d) XML.
- e) ASP.

- RESPOSTA
- LETRA A

- No padrão de arquitetura de software MVC (model-view-controller), model (modelo) é o responsável pela camada de interação com o usuário e realiza a exibição dos dados, que, em sistemas web, se dá por meio do HTML ou do XML.
- Errado
- Certo

- RESPOSTA
- ERRADO

- O padrão MVC separa um aplicativo em três componentes principais: modelo, exibição e controlador; sendo o modelo o componente que manipula e responde à entrada e à interação do usuário.
- Errado
- Certo

- RESPOTA
- ERRADO

- O padrão de arquitetura de software MVC (Model-View-Controller) separa a representação da informação da interação do usuário com ele. O Model consiste basicamente:
 - a) nos dados da aplicação, regras de negócios, lógica e funções
 - b) em qualquer saída de representação dos dados, como uma tabela ou um diagrama
 - c) em interagir diretamente com os usuários finais através de qualquer interface com o usuário
 - d) em fazer a mediação da entrada, convertendo-a em comandos
 - e) em permitir o compartilhamento de recursos numa larga escala

- RESPOSTA
- LETRA A

- É na camada View, da arquitetura MVC, que é realizada toda a lógica do aplicativo, como, por exemplo, as validações e regras de negócios.
- Errado
- Certo

- RESPOSTA
- ERRADO

- Na arquitetura MVC, a definição de propriedades e atributos de um programa ocorrem na camada Model.
- Errado
- Certo

- RESPOSTA
- CERTO

- É uma característica do modelo MVC, EXCETO:
- a) ser um padrão de arquitetura de software.
- b) facilitar o desenvolvimento e a manutenção do software.
- c) permitir total independência entre a forma de acesso aos dados, a lógica de negócios e a lógica de apresentação.
- d) ser conhecida como uma arquitetura de camadas.
- e) impossibilitar a reutilização do código quando utilizado com linguagem orientada a objetos.

- RESPOSTA
- LETRA E

- Modelo MVC significa
 - a) Modo-View-Construtor.
 - b) Modelo-View-Controlador.
 - c) Modelo-Versão-Case.
 - d) Módulo-Verificador-Controlador.
 - e) Medida-Virtual-Concepção.

- RESPOSTA
- LETRA B

- A definição de que um sistema deve ser desenvolvido em três níveis é feita pelo padrão de projeto
- a) MVC (Model View Controller).
- b) MVC-Dev (Model Value Constructive Development).
- c) TMS (Time Milestones Setting).
- d) PMC (Project Main Controller).
- e) MCA (Model Classes Assignment).

- RESPOSTA
- LETRA A

- No padrão MVC, uma mesma lógica de negócios pode ter diferentes interfaces de usuário.

Alternativas

- Certo
- Errado

- RESPOSTA
- CERTO

- No modelo MVC (Model-View-Controller), a camada controller possui uma grande importância, não menos importante em relação as outras camadas, View e Model, desta forma, assinale a alternativa que apresenta qual o objetivo da camada controller neste modelo.
- (A) Controlar o armazenamento de dados persistentes
- (B) Controlar o gerenciamento de memória do software
- (C) Controlar o fluxo da aplicação, recebendo entradas do usuário e atualizando o modelo e a visualização correspondentes
- (D) Controlar o fluxo da aplicação, mapeando as unidades de rede nos servidores de banco de dados

- RESPOSTA
- LETRA C

- O desenvolvimento em três camadas já vem sendo aplicado há alguns anos, um dos modelos mais utilizados hoje em dia é o MVC (Model-View-Controller), sobre este modelo assinale a alternativa que apresenta qual é a responsabilidade da camada Model:
- (A) A camada responsável por gerenciar os recursos de entrada do usuário
- (B) A camada responsável por integrar a entrada de dados
- (C) A camada responsável por renderizar os dados de saída para o usuário
- (D) A camada que contém a lógica de negócios e os dados da aplicação

- RESPOSTA
- LETRA D

Arquitetura de Software

- Dúvidas?